

PIANO TECNICO E IMPLEMENTATIVO

Passione Fantascienza

Architettura del prototipo e percorso di evoluzione su GitHub Pages

Documento tecnico · proposto da Giordano Sognatore

26 settembre 2026

*Dalla conversazione alla rivista: trasformare una community Telegram in un archivio vivo,
pubblico e riconoscibile della fantascienza.*

1. Obiettivo tecnico

Realizzare un sito editoriale static-first per Passione Fantascienza, pubblicato su GitHub Pages e gestito da repository Git. La prima versione deve funzionare come prototipo dimostrativo, ma la struttura deve essere sufficientemente solida da evolvere nel sito reale senza una riscrittura completa.

Scelta architetturale — Separare fin dall'inizio contenuti, presentazione, tassonomia e automazione di build. Gli articoli restano dati editoriali portabili; il tema grafico e i componenti possono evolvere senza riscriverli.

Vincoli e requisiti

- hosting iniziale a costo minimo, preferibilmente GitHub Pages;
- contenuti versionati in Git e scritti soprattutto in Markdown/MDX;
- ottima resa mobile, perché una quota importante del traffico arriverà da Telegram;
- URL stabili, sitemap, RSS e metadata adatti all'indicizzazione;
- ricerca interna senza server o database;
- possibilità di aggiungere collaboratori mediante pull request o workflow editoriali;
- nessuna dipendenza iniziale da un backend proprietario;
- possibilità futura di integrare servizi esterni senza migrare i contenuti.

2. Perché GitHub Pages è adatto

GitHub Pages può pubblicare file statici e può usare GitHub Actions per costruire il sito con generatori statici diversi da Jekyll. La documentazione GitHub specifica inoltre che Pages non esegue linguaggi server-side come PHP, Ruby o Python: questa non è una limitazione problematica per il progetto iniziale, perché blog, archivi, categorie, ricerca statica, RSS, sitemap e pagine autore possono essere prodotti in fase di build.

Funzione	Compatibilità con Pages	Soluzione prevista
Landing / magazine	Piena	HTML/CSS/JS generati da Astro
Articoli Markdown	Piena	Astro Content Collections
Categorie, tag, autori	Piena	Route statiche generate in build
Ricerca	Piena	Pagefind post-build
RSS	Piena	@astrojs/rss
Sitemap	Piena	@astrojs/sitemap
Dominio personalizzato	Piena	DNS + configurazione Pages
HTTPS	Piena	HTTPS supportato da GitHub Pages

Funzione	Compatibilità con Pages	Soluzione prevista
Commenti	Esterna	Opzionale: servizio basato su GitHub Discussions o altro provider
Newsletter	Esterna	Provider esterno, collegato dal sito
Login / profili utenti	Non nativo	Backend/API esterna solo se un giorno necessario
Database dinamico	Non nativo	Servizio esterno o migrazione ibrida, mantenendo i contenuti statici

3. Stack raccomandato

Livello	Tecnologia	Motivazione
Hosting	GitHub Pages	Semplice, già familiare, perfetto per output statico e dominio personalizzato.
Repository / workflow	Git + GitHub	Versionamento, issue, pull request, revisioni e storico editoriale.
Generatore	Astro	Produce siti statici prerenderizzati, adatto a contenuti editoriali e componenti riutilizzabili.
Contenuti	Markdown + MDX	Leggibili, portabili, facili da revisionare e conservare.
Modello contenuti	Astro Content Collections	Schema, validazione e tipizzazione dei metadati editoriali.
Stili	CSS nativo + design tokens	Controllo completo, bundle ridotto, meno dipendenze; Tailwind resta opzionale.
Ricerca	Pagefind	Genera un indice statico dopo la build; nessun server di ricerca.
Feed	@astrojs/rss	Feed aggiornato automaticamente a ogni build.
Sitemap	@astrojs/sitemap	Generazione automatica delle URL del sito.
Deploy	GitHub Actions + action Astro	Build e pubblicazione automatiche a ogni merge/push previsto.

Confronto sintetico tra generatori

Opzione	Vantaggi	Svantaggi nel progetto	Valutazione
Astro	Ottimo per contenuti, componenti moderni, HTML statico, integrazioni ufficiali, buona ergonomia frontend.	Richiede ecosistema Node in sviluppo/build, anche se il deploy può essere delegato a GitHub Actions.	Raccomandato
Hugo	Molto veloce, binario singolo, eccellente per siti statici e grandi archivi.	Templating e personalizzazioni frontend meno immediati per chi vuole componenti moderni.	Ottima alternativa
Jekyll	Storicamente integrato con GitHub Pages, Markdown e Liquid.	Ruby e workflow/template meno adatti a un progetto nuovo con forte componente UI.	Possibile ma non preferito

La scelta raccomandata è Astro. Il problema di dover installare Node sul computer locale non influenza l'hosting: la build di produzione può avvenire interamente su GitHub Actions. Per lo sviluppo locale è comunque utile un runtime Node; in alternativa si può usare un ambiente di sviluppo cloud o lavorare su file semplici affidando la build al repository.

4. Architettura logica

La pipeline proposta è volutamente semplice:

Markdown / dati → Astro → HTML statico → Pagefind → artifact di GitHub Pages → deploy HTTPS

Il repository contiene sia il codice del sito sia i contenuti editoriali. Le modifiche entrano su branch o pull request; la build valida i contenuti e genera l'output. Soltanto una build riuscita viene pubblicata.

Struttura indicativa del repository

```

passione-fantascienza/
├── .github/
│   └── workflows/
│       └── deploy.yml
├── public/
│   ├── favicon.svg
│   └── media/
├── src/
│   ├── assets/
│   ├── components/
│   ├── content/
│   │   ├── articoli/
│   │   ├── autori/
│   │   ├── risorse/
│   │   └── eventi/
│   ├── layouts/
│   └── pages/

```

```

|   | styles/
|   | content.config.ts
|   | astro.config.mjs
|   | package.json
|   | package-lock.json

```

5. Modello dei contenuti

I contenuti devono essere modellati come dati editoriali, non come pagine HTML scritte a mano. Astro Content Collections consente di definire collezioni e schemi per validare i metadati: se un articolo manca di un campo obbligatorio o usa un valore non ammesso, la build può fallire prima della pubblicazione.

Collezione “articoli”

Campo	Tipo	Uso
title	stringa	Titolo editoriale
description	stringa	Abstract per card e SEO
date	data	Data di pubblicazione
updated	data opzionale	Data di aggiornamento sostanziale
author	riferimento autore	Firma e collegamento alla pagina autore
section	enum	libri / cinema-tv / community / dossier / autori / risorse
type	enum	recensione / intervista / editoriale / guida / news / community / lista
tags	array	Indicizzazione trasversale
cover	immagine	Card, articolo e Open Graph
featured	booleano	Contenuto evidenziabile in home
telegramUrl	URL opzionale	Discussione o invito correlato
draft	booleano	Esclusione dalla pubblicazione

Esempio di articolo Markdown

```
---
```

```

title: "Isaac Asimov e il problema della sicurezza delle AI"
description: "Dalle Tre Leggi ai sistemi contemporanei: cosa resta utile e cosa cambia."
date: 2026-10-04
author: "giordano-sognatore"
section: "dossier"
type: "approfondimento"
tags: ["Asimov", "AI", "robotica"]
cover: "/media/articoli/asimov-ai.webp"
featured: true
draft: false
---
```

Testo dell'articolo...

Altre collezioni

- **autori:** nome, slug, bio breve/estesa, foto, link, ruolo, libri/opere selezionate;
- **risorse:** titolo, categoria, URL, descrizione, lingua, stato/ultima verifica;
- **eventi:** titolo, data/ora, tipo, luogo o URL, organizzatore, collegamenti;
- **eventuali opere:** metadati bibliografici utili a recensioni e pagine autore, se l'archivio cresce abbastanza da giustificarlo.

6. URL e tassonomia

Tipo	Schema URL
Articolo	/articoli/<slug>/
Sezione	/libri/ · /cinema-tv/ · /community/ · /dossier/
Autore	/autori/<slug>/
Tag	/tag/<slug>/
Rubrica	/rubriche/<slug>/
Risorsa	/risorse/ oppure pagina dedicata se necessario
Ricerca	/cerca/

Gli slug devono restare stabili dopo la pubblicazione. Se un titolo cambia, l'URL non va modificato salvo necessità forte; in caso contrario servono redirect, che su hosting statico vanno pianificati con attenzione.

7. Componenti del prototipo

Componente	Responsabilità
Header / Navigation	Brand, sezioni, ricerca, accesso Telegram.
HeroArticle	Articolo principale della homepage.
ArticleCard	Card riusabile per ultimi articoli, categorie e correlati.
SectionStrip	Riga tematica per Libri, Cinema & TV, Dossier ecc.
CommunityPulse	Box dedicato a "Questa settimana nella community".
AuthorCard	Scheda sintetica dell'autore / ospite.
TelegramCTA	Invito coerente al gruppo, riusato nei punti strategici.
TagList	Navigazione tematica.
SearchDialog / SearchPage	Interfaccia Pagefind.
RelatedContent	Articoli correlati per tag/sezione/autore.
Footer	Info, contatti, policy, RSS, risorse principali.

8. Design system e UX

Il design dovrebbe evitare il cliché "sfondo nero + neon + astronave generica". L'obiettivo è una rivista leggibile, autorevole e riconoscibile, con elementi fantascientifici usati come linguaggio editoriale e non come decorazione invadente.

- **Mobile first.** Layout progettato prima per smartphone, quindi ampliato su desktop.
- **Tipografia leggibile.** Larghezza della colonna testo controllata, interlinea comoda, gerarchia forte di titoli e sottotitoli.
- **Design tokens.** Colori, spaziature, raggi, ombre e dimensioni tipografiche centralizzati in variabili CSS.
- **Componenti riusabili.** Una card o un box community deve avere una singola implementazione coerente in tutto il sito.
- **Accessibilità.** Contrasto, focus da tastiera, alt text, struttura semantica, label esplicite e uso prudente delle animazioni.
- **Immagini responsive.** Formati moderni, dimensioni adeguate, lazy loading dove appropriato e niente immagini enormi per card piccole.

9. Ricerca interna con Pagefind

Pagefind è particolarmente adatto perché indicizza l'HTML statico dopo la build e genera un bundle di ricerca statico. Non richiede un server di ricerca: l'indice viene distribuito insieme al sito e interrogato dal browser.

1. Astro genera il sito nella directory di output.
2. Pagefind scansiona le pagine HTML prodotte.
3. Viene creato l'indice statico e la relativa UI o API client.
4. GitHub Pages pubblica sito e indice nello stesso artifact.

Per il prototipo è sufficiente indicizzare titolo, descrizione e corpo degli articoli. In seguito si possono usare metadati o filtri per categoria, autore e tag.

10. SEO, feed e metadata

Elemento	Implementazione prevista
Title / description	Generati per pagina con fallback globali.
Canonical URL	Basata sulla configurazione site di Astro e sull'URL pubblico.
Open Graph	Titolo, descrizione, immagine e tipo articolo/pagina.
Twitter/X cards	Metadata compatibili con anteprima link.
Sitemap	@astrojs/sitemap durante la build.
RSS	@astrojs/rss per gli articoli pubblici.
robots.txt	Statico, con riferimento alla sitemap.
Structured data	JSON-LD progressivo: Article/Review, Person, BreadcrumbList dove appropriato.
Breadcrumb	Visuale e semantico per archivi, sezioni e articoli.

11. GitHub Actions e deploy

Astro documenta un'Action ufficiale per distribuire un sito statico prerenderizzato su GitHub Pages. GitHub Pages supporta workflow personalizzati; il deploy consigliato usa GitHub Actions e pubblica un artifact statico.

Pipeline proposta

1. Checkout del repository.
2. Installazione dipendenze secondo il lockfile committato.
3. Controlli di qualità: formattazione/lint dove configurati e validazione delle Content Collections.

4. Build Astro.
5. Generazione indice Pagefind sull'output statico.
6. Verifica minima dell'output (esistenza home, sitemap, RSS, indice ricerca).
7. Upload artifact Pages.
8. Deploy nell'environment github-pages.

Regola di pubblicazione — La branch principale è la fonte pubblicabile. Per un piccolo gruppo si può iniziare con merge manuali; crescendo, si proteggono branch e pull request e si richiede almeno una revisione editoriale o tecnica sui contenuti più importanti.

12. Workflow editoriale su GitHub

Livello di maturità	Metodo
Prototipo	Giordano modifica contenuti e codice direttamente su branch dedicati; merge dopo verifica.
Piccola redazione	Ogni contributo passa da pull request; template PR chiede titolo, categoria, fonti, immagini e disclosure.
Collaboratori non tecnici	Un redattore trasforma i testi ricevuti in Markdown oppure si introduce una UI CMS Git-based.
Redazione più ampia	Ruoli GitHub, code owners per aree, label editoriali, issue template e calendario condiviso esterno.

Non introdurrei un CMS visuale nel prototipo. Prima conviene validare il modello editoriale. Se più persone non tecniche iniziano a pubblicare con frequenza, un CMS Git-based può essere aggiunto senza cambiare il formato dei contenuti.

13. Strategie per immagini e media

- Conservare nel repository soltanto media necessari e ottimizzati; evitare file originali enormi.
- Definire proporzioni standard per cover articolo, avatar autore e immagini inline.
- Usare nomi file stabili e descrittivi, preferibilmente WebP/AVIF quando compatibile con il workflow.
- Aggiungere alt text obbligatorio nel modello editoriale.
- Per video e contenuti pesanti preferire embed/collegamenti a piattaforme esterne invece di archiviare i file nel repository.
- Documentare licenza e provenienza delle immagini quando non sono prodotte internamente.

14. Integrazione con Telegram

L'integrazione iniziale dovrebbe essere semplice e robusta: link profondi al gruppo/canale quando disponibili, pulsanti di invito e rimandi editoriali. Embed o widget specifici possono essere aggiunti in una fase successiva se migliorano davvero l'esperienza e rispettano privacy e prestazioni.

- CTA Telegram persistente ma non invasiva in header/mobile menu e footer;

- CTA contestuale a fine articolo: “continua la discussione nella community”;
- campo telegramUrl negli articoli quando esiste una discussione o un messaggio correlato;
- pagina Community che spiega regole, tono e modalità di ingresso;
- rubrica “Questa settimana...” come ponte editoriale stabile.

15. Privacy, analytics e servizi esterni

Per il prototipo conviene minimizzare i servizi terzi. Il sito statico può funzionare senza account utente, cookie applicativi o database. Se si aggiunge analytics, newsletter o commenti, bisogna valutare privacy, cookie, informative e carico di moderazione prima di integrarli.

Servizio	Prototipo	Evoluzione
Analytics	Opzionale o assente	Aggiungere uno strumento essenziale e privacy-oriented se serve misurare traffico e conversioni.
Newsletter	Solo CTA “in arrivo” o nessuna	Provider esterno quando esiste un ritmo editoriale sufficiente.
Commenti web	No	Valutare soltanto se aggiungono valore rispetto alla discussione Telegram.
Form collaborazioni	Email/link semplice	Form esterno o serverless quando aumentano le richieste.
Account utenti	No	Introdurre solo se emerge un caso d'uso reale non risolvibile con Telegram.

16. Sicurezza e affidabilità

- abilitare HTTPS e, con dominio personalizzato, configurare correttamente DNS e verifica del dominio;
- proteggere la branch di pubblicazione quando iniziano a operare più persone;
- non inserire segreti nel repository o nel codice client; usare GitHub Secrets solo per integrazioni che ne abbiano necessità;
- mantenere dipendenze aggiornate in modo controllato e committare il lockfile;
- eseguire build automatica su pull request per intercettare errori di schema, link o compilazione prima del merge;
- considerare backup/export periodici del repository: l'intero sito è già versionato, quindi il ripristino è semplice finché media e configurazioni sono nel repository.

17. Performance e qualità

Controllo	Obiettivo
HTML statico	Ridurre dipendenza da JavaScript per la lettura dei contenuti.

Controllo	Obiettivo
Bundle JS	Caricare solo componenti interattivi realmente necessari.
Immagini	Dimensioni responsive, compressione e lazy loading per contenuti non above-the-fold.
CSS	Foglio coerente, ridotto, con design tokens e componenti.
Lighthouse / Web Vitals	Usare come verifica, non come obiettivo cosmetico; correggere regressioni significative.
Link interni	Controllo automatico periodico e gestione dei link esterni non più validi nella sezione Risorse.
Accessibilità	Test tastiera, semantic HTML, label, focus, contrasto e alt text.

18. Implementazione del prototipo: scope

Modulo	Incluso nel prototipo
Home	Hero, ultimi articoli, sezioni, CommunityPulse, autore ospite, CTA Telegram.
Articolo	Layout completo, autore, data, tag, related content, CTA Telegram.
Archivi	Tutti gli articoli + pagine di sezione e tag.
Autori	Indice e singola pagina autore.
Community	Presentazione gruppo + rubrica dimostrativa.
Risorse	Piccolo archivio ragionato.
Ricerca	Pagefind.
SEO tecnico	Title/description, canonical, OG, sitemap, RSS, robots.
Responsive	Smartphone, tablet, desktop.
Deploy	GitHub Actions → GitHub Pages.

Contenuti dimostrativi

- 6–10 articoli campione distribuiti tra libri, cinema/TV, dossier e community;
- 1 articolo lungo “Questa settimana in Passione Fantascienza”;
- 2 pagine autore, di cui una può essere Giordano Sognatore usata come modello tecnico/editoriale;
- 1 pagina “Collabora con noi”;
- 1 pagina Risorse con 8–15 link strutturati;
- tag e categorie sufficienti a mostrare navigazione e ricerca.

19. Evoluzione successiva

Quando emerge il bisogno	Evoluzione possibile
Molti redattori non tecnici	CMS Git-based o interfaccia editoriale dedicata.
Calendario ricco	Collezione eventi + viste mensili/filtri; eventuale feed iCal.
Molte collaborazioni	Form strutturato e workflow issue/CRM leggero.
Commenti richiesti	Sistema esterno moderabile; valutare sovrapposizione con Telegram.
Newsletter sostenibile	Provider esterno con iscrizione conforme alla privacy.
Database di opere/autori molto ampio	Dati strutturati dedicati o servizio esterno mantenendo il frontend Astro.
Funzioni con login	Backend/API esterna o piattaforma ibrida; il corpus Markdown può restare invariato.

20. Criteri di accettazione del prototipo

- deploy automatico su GitHub Pages completato da repository pulito;
- nessun errore di build con contenuti validi; errore esplicito su frontmatter non conforme;
- navigazione completa e usabile da smartphone;
- ricerca che restituisce gli articoli campione;
- RSS e sitemap generati e raggiungibili;
- ogni articolo ha metadata social, autore, categoria/tag e CTA Telegram;
- homepage caricabile senza dipendere da servizi esterni critici;
- layout accessibile con tastiera e senza overflow alle principali larghezze;
- documentazione README sufficiente a clonare, modificare contenuti e pubblicare.

21. Decisioni tecniche da confermare prima della build

1. Repository GitHub e account/organizzazione che ne avrà la titolarità.
2. URL del gruppo/canale Telegram da usare nelle CTA e natura pubblica o privata del gruppo.
3. Nome visuale definitivo e presenza di un logo già approvato.

4. Dominio: nessuno nella fase demo oppure dominio personalizzato al lancio.
5. Categorie iniziali e nomi delle rubriche.
6. Chi può approvare il merge sulla branch pubblicata.
7. Policy minima su immagini, citazioni della chat, recensioni e contenuti sponsorizzati.

22. Conclusione tecnica

L'architettura Astro + Markdown/MDX + Pagefind + GitHub Actions + GitHub Pages è adeguata sia a un prototipo sia a un magazine editoriale di dimensioni significative, finché il nucleo del prodotto resta la pubblicazione di contenuti. Offre prestazioni elevate, versionamento completo, costi di hosting iniziali molto bassi e una via di crescita progressiva.

La scelta più importante è non introdurre prematuramente backend, database e CMS complessi. Il progetto può iniziare con un sistema interamente statico e aggiungere servizi esterni solo quando un'esigenza reale li rende necessari. In questo modo la complessità segue la crescita della community, anziché precederla.

Appendice A · Workflow GitHub Pages di riferimento

```
name: Deploy to GitHub Pages

on:
  push:
    branches: [main]
    workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: withastro/action@v6
      # eventuale step Pagefind integrato nello script di build

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - id: deployment
        uses: actions/deploy-pages@v5
```

Nota: le versioni delle Action vanno mantenute aggiornate. In implementazione si userà la configurazione ufficiale Astro/GitHub corrente e si integrerà Pagefind nello script di build in modo che l'indice sia compreso nell'artifact finale.

Appendice B · Riferimenti tecnici verificati

Riferimenti consultati

Verifica aggiornata al 26 settembre 2026. Le fonti sono indicate per documentare i modelli editoriali o le capacità tecniche richiamate nel testo.

- GitHub Docs — Creating a GitHub Pages site: <https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site>
- GitHub Docs — Using custom workflows with GitHub Pages: <https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages>
- GitHub Docs — HTTPS for GitHub Pages: <https://docs.github.com/en/pages/getting-started-with-github-pages/securing-your-github-pages-site-with-https>
- GitHub Docs — Custom domains: <https://docs.github.com/en/pages/configuring-a-custom-domain-for-your-github-pages-site/managing-a-custom-domain-for-your-github-pages-site>
- Astro Docs — Deploy to GitHub Pages: <https://docs.astro.build/en/guides/deploy/github/>
- Astro Docs — Content Collections: <https://docs.astro.build/en/guides/content-collections/>
- Astro Docs — RSS: <https://docs.astro.build/en/recipes/rss/>
- Astro Docs — Sitemap: <https://docs.astro.build/en/guides/integrations-guide/sitemap/>
- Pagefind — Getting Started: <https://pagefind.app/docs/>